

Enums java

Juan Ignacio Raggio

May 3, 2025

Contents

1	Que son?	1
1.1	Poder de los enums	2
1.1.1	Mejor forma:	3
1.1.2	Metodos que vienen de la superclass de los enums . . .	4
1.2	Cuando usar enums?	4
2	Metodos abstractos dentro de enums	4
	ENUMS	
2.1	Una clase que implementa a otra tiene que implementar todos los metodos o ponerlo como abstracto	4
2.2	En el fondo cada enum se esta guardando un puntero a funcion	4
2.3	Hay que poner todas las operaciones posibles dentro del enum	
	TIP	5
3	Classpath	5
4	Otro beneficio de usar Enums en Java	5
	EFICIENCIA:EFFICIENCY	
5	compareTo(), equals(), hashCode() en Enums	5

1 Que son?

- Su forma mas simple se usan como los enums de C -> **contenedores de constantes**
 - La diferencia con los enums de C es que en java SI se chequea el tipo

```
public static final int SUCCESS = 0;
public static final int GOOD = 1;
public static final int REGULAR = 2;
public static final int BAD = 3;
```

- La siguiente es la forma correcta:

```
public enum Rating {
    SUCCESS, GOOD, REGULAR, BAD
}
```

Java nos asegura de crear si o si una sola instancia de los mismos

1.1 Poder de los enums

- Pueden tener metodos de clase
- Tambien pueden tener metodos de instancia

```
public enum Rating {

    SUCCESS, GOOD, REGULAR, BAD

    public static String toString(Rating rating) {
        if (rating == null) return "";
        switch (rating) {
            case SUCCESS -> "Success";
            case GOOD -> "Good";
            case REGULAR -> "Regular";
            case BAD -> "Bad";
        }
    }
}
```

// Como puedo tener metodos de instancia, podemos hacer:

```
@Override
public String toString(Rating rating) {
    return switch (this) {
        case SUCCESS -> "Success";
        case GOOD -> "Good";
        case REGULAR -> "Regular";
    };
}
```

```

        case BAD -> "Bad";
    };
}

}

```

1.1.1 Mejor forma:

```

public enum Rating {
    // Forma de invocacion al constructor
    SUCCESS("Success", 10),
    GOOD("Good", 7),
    REGULAR("Regular", 5),
    BAD("Bad", 2);

    private final String description;
    private final int value;

    // Constructor
    Rating (String description, int value) {
        this.description = description;
        this.value = value;
    }

    @Override
    public String toString() { return description; }

    @Override
    public int intValue() { return value; }
}

// Como tener una variable de tipo succes
main {
    Rating good = Rating.GOOD;
    sout(good); // Imprime Good
    sout(good.intValue()); // Imprime 7
    sout(Rating.GOOD); // Tambien es valido
}

```

1.1.2 Metodos que vienen de la superclass de los enums

1. `.ordinal()` -> Me retorna el "orden"
2. `.valueOf()` -> Recibe un String y me da la instancia asociada

- Ejemplo:

```
Rating success = Rating.valueOf("SUCCESS"); -> Me retorna la instancia de S
Rating.valueOf("Success"); -> IllegalArgumentException pues es case sensitiv
```

3. `values()` -> Me retorna un arreglo con los enums en orden

1.2 Cuando usar enums?

- Cuando se que solo quiero una instancia de una cantidad de objetos fija

2 Metodos abstractos dentro de enums ENUMS

2.1 Una clase que implementa a otra tiene que implementar todos los metodos o ponerlo como abstracto

- En caso de que ponga por lo menos un metodo como abstracto, recordemos que si o si pase a ser abstracta

2.2 En el fondo cada enum se esta guardando un puntero a funcion

```
public enum Operation {
    ADD("+") {
        @Override
        public double apply(double op1, double op2) {
            return op1 + op2;
        }
    },
    SUBTRACT("-") { ... }, MULTIPLY("*") { ... }, DIVIDE("/") { ... };

    private final String symbol;

    Operation(String symbol) { this.symbol = symbol; }
```

```

public abstract double apply(double op1, double op2);

public static double evaluate(String op1, String op2, String op) {
    double n1 = Double.parseDouble(op1); double n2 = Double.parseDouble(op2);
    for (Operation instance : values()) {
        if (instance.symbol.equals(op)) {
            return instance.apply(n1, n2);
        }
    }
    throw new IllegalArgumentException();
}

```

2.3 Hay que poner todas las operaciones posibles dentro del enum TIP

3 Classpath

- Si quiero usar una clase que esta en otro lado la tengo que agregar al classpath

4 Otro beneficio de usar Enums en Java EFICIENCIA:EFFICIENCY

- Note that when using an enumeration type as the type of a set or as the type of the keys in a map, specialized and efficient set and map implementations are available.

5 compareTo(), equals(), hashCode() en Enums

- Todos estos metodos ya estan implementados por la superclass Enum
- Ademas estan prefijados con la palabra clave `final`
- En conclusion NO pueden ser sobreescritos, por lo que el orden natural de los enums viene dado por el orden en el que se definen dentro del mismo